AA274A Homework 2

Name: Colin Skinner
SUNet ID: 06525671

Collaborators: None
Use of generative AI: LaTeX matrix syntax

Problem 1: Trajectory Optimization

(i)

Decision Variables that the optimizer can change:

1. State: x_k, y_k, θ_k for $k = 0, \dots, N$
2. Control: v_k, ω_k for $k = 0, \dots, N-1$
3. Final time t_f

Discretized cost function to be minimized:

$$J = \frac{t_f}{N} \sum_{k=0}^{N-1} \left(\alpha + v_k^2 + \omega_k^2\right) \tag{1}$$

Initial Conditions:

$$x(0) = 0 \tag{2}$$
$$y(0) = 0 \tag{3}$$
$$\theta(0) = \pi/2 \tag{4}$$

Final Conditions:

$$x(t_f) = 5 \tag{5}$$
$$y(t_f) = 5 \tag{6}$$
$$\theta(t_f) = \pi/2 \tag{7}$$

Collision Avoidance Constraint:

$$x_{obstacle} = 2.5 \tag{8}$$
$$y_{obstacle} = 2.5 \tag{9}$$
$$r_{obstacle} = 0.3 \tag{10}$$
$$r_{ego} = 0.1 \tag{11}$$
$$\sqrt{(x_k - x_{obstacle})^2 + (y_k - y_{obstacle})^2} - (r_{ego} + r_{obstacle}) \geq 0 \quad \forall k \in (0, N) \tag{12}$$

Dynamics Constraints:

$$x_{k+1} = x_k + \frac{t_f}{N} v_k \cos(\theta_k) \tag{13}$$
$$y_{k+1} = y_k + \frac{t_f}{N} v_k \sin(\theta_k) \tag{14}$$
$$\theta_{k+1} = \theta_k + \frac{t_f}{N} \omega_k \tag{15}$$
$$\tag{16}$$

Bounds:

$$0 \leq t \leq t_f \tag{17}$$

$$v_{max} = 0.5 \tag{18}$$
$$0.01 \leq v(t) \leq v_{max} \tag{19}$$

$$\omega_{max} = 1 \tag{20}$$
$$|\omega(t)| \leq \omega_{max} \tag{21}$$

(ii)

Graded by the autograder; nothing to include.

(iii)(a)

Answer: $v^* = \sqrt{\alpha}$

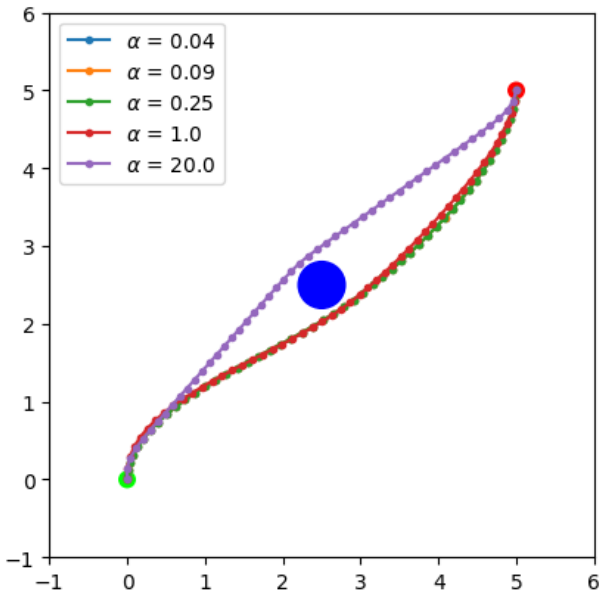
$$J(v) = L(\alpha/v + v) \tag{22}$$

So we minimize by taking the derivative with respect to v

$$\frac{\partial J}{\partial v} = \frac{\partial}{\partial v}(L(\alpha/v + v)) = 0 \tag{23}$$
$$= L\frac{\partial}{\partial v}(\alpha/v + v) \tag{24}$$
$$= \frac{\partial}{\partial v}(\alpha/v + v) \tag{25}$$
$$= -\frac{\alpha}{v^2} + 1 \tag{26}$$
$$\therefore \tag{27}$$
$$\frac{\alpha}{v^2} = 1 \tag{28}$$
$$v^2 = \alpha \tag{29}$$
$$v = \sqrt{\alpha} \tag{30}$$

To have the velocity constraint be active, $\alpha \geq v_{max}^2$ so $\alpha \geq 0.25$. Increasing it will make the optimum velocity faster, so the actuator commands it to the maximum velocity.

(iii)(b)



Answer:

$\alpha = 0.04$: median speed (0.198 m/s), prediction (0.2 m/s);
$\alpha = 0.25$: median speed (0.494 m/s), prediction (0.5 m/s)

Problem 2: Numpy and Class Inheritance

(i)

Graded by the autograder; nothing to include.

(ii)(a)

Equation:

$$\boldsymbol{X}_{t+1} = \bar{A}\boldsymbol{X}_t + \bar{B}u_t \tag{31}$$

State:

$$\boldsymbol{X}_t = \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_{R-1} \end{bmatrix} \in \mathbb{R}^{4R} \tag{32}$$

Matrices:

$$\bar{A} = \begin{bmatrix} A_0 & 0 & 0 & ... & 0 \\ 0 & A_1 & 0 & ... & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & ... & A_{R-1} \end{bmatrix} \in \mathbb{R}^{4R\times 4R} \tag{33}$$

$$\bar{B} = \begin{bmatrix} B_0 \\ B_1 \\ \vdots \\ B_{R-1} \end{bmatrix} \in \mathbb{R}^{4R\times 2} \tag{34}$$

(ii)(b)

Graded by the autograder; nothing to include.

(iii)(a)

Answer: $\Delta t = 0.1$: $x(T) = 0.4500$; $\Delta t = 0.01$: $x(T) = 0.4950$

(iii)(b)

Proving:

$$x_k = \frac{a\Delta t^2 K(K-1)}{2} \tag{35}$$

Starting with v_k:

$$v_k = v_{k-1} + a\Delta t \tag{36}$$

$$\therefore \tag{37}$$

$$v_1 = v_0 + a\Delta t = a\Delta t \tag{38}$$

$$v_2 = v_1 + a\Delta t = 2a\Delta t \tag{39}$$

$$\vdots \tag{40}$$

$$v_k = ka\Delta t \tag{41}$$

To derive x_K, we note the following series identities:

$$\sum_{k=1}^{K} k = \frac{K}{2}(K+1) \tag{42}$$

$$\sum_{i=1}^{K}(x_i - x_{i-1}) = x_K - x_0 \tag{43}$$

$$\tag{44}$$

Then back to x_K:

$$x_k = x_{k-1} + v_{k-1}\Delta t \tag{45}$$

$$x_k - x_{k-1} = v_{k-1}\Delta t \tag{46}$$

Finally, summing both sides and using the above equations 42 and 43

$$\sum_{k=1}^{K}(x_k - x_{k-1}) = \sum_{k=1}^{K}(v_{k-1}\Delta t) \tag{47}$$

$$x_K - x_0 = \Delta t \sum_{k=1}^{K} v_{k-1} \tag{48}$$

$$x_K = \Delta t \sum_{k=1}^{K} v_{k-1} \tag{49}$$

$$= \Delta t \sum_{k=1}^{K}((k-1)a\Delta t) \tag{50}$$

$$= a\Delta t^2 \sum_{k=1}^{K}(k-1) \tag{51}$$

$$= a\Delta t^2 \left[\sum_{k=1}^{K} k - \sum_{k=1}^{K} 1\right] \tag{52}$$

$$= a\Delta t^2 \left[\frac{K}{2}(K+1) - K\right] \tag{53}$$

$$= a\Delta t^2 \left[\frac{K^2 + K - 2K}{2}\right] \tag{54}$$

$$= a\Delta t^2 \left[\frac{K^2 - K}{2}\right] \tag{55}$$

$$= a\Delta t^2 \left[\frac{K(K-1)}{2}\right] \tag{56}$$

$$\tag{57}$$

$$x_K = \frac{a\Delta t^2 K(K-1)}{2} \tag{58}$$

$$\tag{59}$$

For the second part, constant acceleration gives:

$$x(t) = at^2/2 \tag{60}$$
$$\tag{61}$$

so:

$$x(T) - x_k = aT^2/2 - \frac{a\Delta t^2 K(K-1)}{2} \tag{62}$$
$$= \frac{aT(K\Delta t) - aT\Delta t(K-1)}{2} \tag{63}$$
$$= \frac{aT\Delta t(K - K + 1)}{2} \tag{64}$$
$$\tag{65}$$
$$x(T) - x_k = \frac{aT\Delta t}{2} \tag{66}$$
$$\tag{67}$$

Using this formula gives:

$\Delta t = 0.01$: $x(T) - x_k = 0.005$; $\Delta t = 0.1$: $x(T) = 0.05$ which matches the error in the output below

dt = 0.10: Euler x(T) = 0.4500, exact = 0.5000, error = 0.0500
dt = 0.01: Euler x(T) = 0.4950, exact = 0.5000, error = 0.0050

Table 1: Python output for error

Euler integration uses the velocity/derivative at the *start* of the time interval. If the derivatives increase during that time interval, then the summation will underestimate the total. For integrating $v(t)$, since acceleration is constant, then there will be no error. But for $x(t)$, it integrates to have the error proportional to Δt.

Problem 3: LQR with Gain Scheduling

(i)

Graded by the autograder; nothing to include.

(ii)(a)

Answer: $K_j \in \mathbb{R}^{m \times n}$ where m is the input dimensions and n is the state dimensions because:

$$K = R^{-1}B^T P \ \in \mathbb{R}^{m \times m} \ \mathbb{R}^{m \times n} \ \mathbb{R}^{n \times n} \tag{68}$$

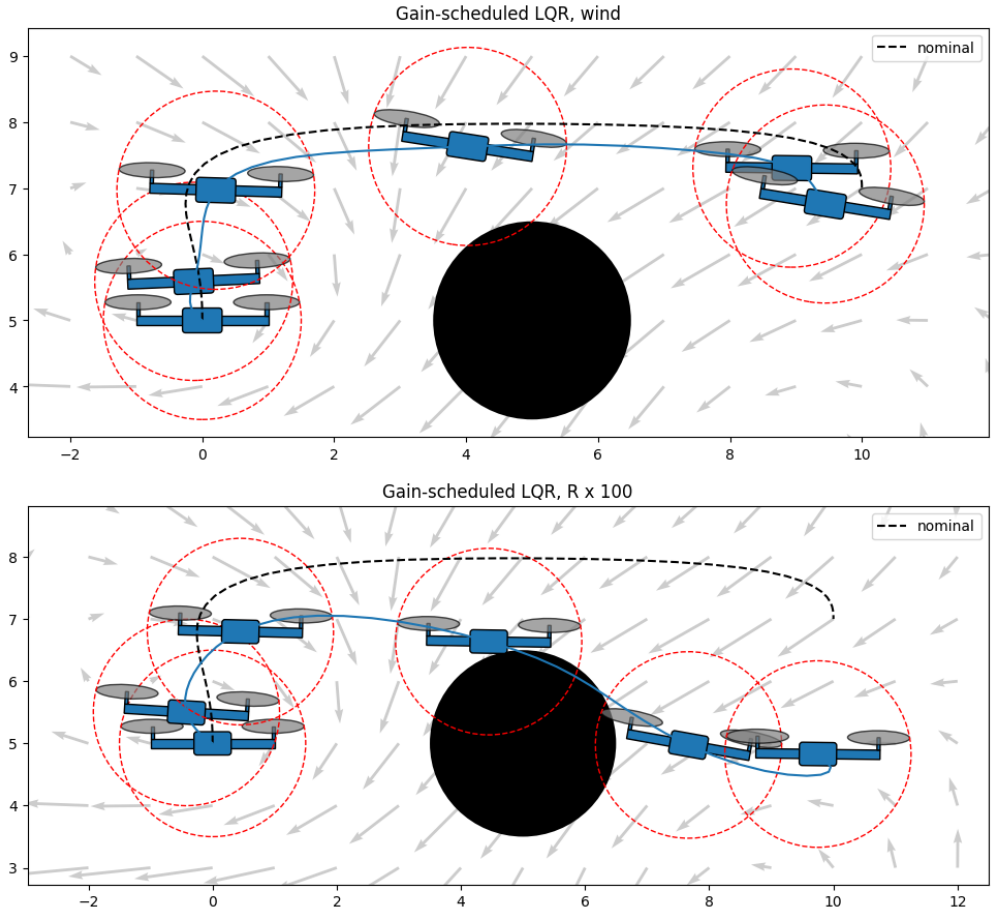
(ii)(b)

Answer: Scaling Q up will penalize the error in the state more, so it try harder to keep the state error at 0 and therefore will tend to track the trajectory closer (but the higher effort results in higher control effort too).

(ii)(c)

Answer: Open loop: (8.188 m); time-indexed LQR: (0.603 m); hover LQR: (0.608 m); nearest-state LQR: (10.348 m)

(iii)(a)



Answer: Having a higher R weights the input more, so the second case tries not to have as much feedback correction (compared to the nominal input), which makes it succumb to the wind more and be pushed away from the optimal path.

(iii)(b)

The maximum pitch is reported as $\phi = 0.327$ rad, and according to the Jacobian function, the linearized matrices are the following:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -\frac{C_{d,v}}{m} & 0 & 0 & -\frac{(T_1+T_2)\cos\phi}{m} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -\frac{C_{d,v}}{m} & -\frac{(T_1+T_2)\sin\phi}{m} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -\frac{C_{d,\phi}}{I_{yy}} \end{bmatrix} \tag{69}$$

$$B = \begin{bmatrix} 0 & 0 \\ -\frac{\sin\phi}{m} & -\frac{\sin\phi}{m} \\ 0 & 0 \\ \frac{\cos\phi}{m} & \frac{\cos\phi}{m} \\ 0 & 0 \\ -\frac{l}{I_{yy}} & \frac{l}{I_{yy}} \end{bmatrix} \tag{70}$$

We can loosely approximate $\cos(\phi) = 0.947 \approx 1$, and we get

$$\frac{\partial v_x}{\partial \phi} \approx -\frac{(T_1 + T_2)}{m} \tag{71}$$

$$\frac{\partial v_y}{\partial T_1} \approx \frac{\partial v_y}{\partial T_2} \approx 1/m \tag{72}$$

This matters because the resulting approximations for these partial derivatives are only about 5% different than what a hovering drone experiences (from ϕ being relatively small the entire flight).

If there is fast lateral acceleration needed, according to the A matrix, the drone will need to tilt to increase ϕ so that $\frac{\partial v_x}{\partial \phi} \neq 0$, and since the above approximations are based on a small-ϕ assumption, it will not hold anymore.

(iii)(c)

The drone hovers near its current state because it does not move far enough during the first timesteps to reach the next ones, so the reference trajectory is a previous one. Since there exists one set of gains for each step, if the algorithm restricted its lookup so that it can only chose states that are forward in time, then it would not be stuck at previous (stale) ones.

The indices are $[0, 1, 1, 2, 2, 3, 3, 3, 3, 2, 0, 0, 0, 0, 0, \ldots]$, showing that after a certain point, it only sticks to the first time index.